APPLIED MATH 3

WEEK 1

INTRO TO MOVEMENT

Review & Basics – Input and Movement in Unreal

INTRODUCTION TO PLAYER CLASSES IN UNREAL

- In Unreal, several classes are available for creating game objects within the world. We will examine each in more detail.
  - Actor: Non-player/AI controlled game objects that exist in the scene.
  - Pawn: Simplest Player/AI controlled game object. Minimal functionality. This is an Actor with additional tools for interacting with Controllers.
  - Character: This is simply a Pawn that is pre-configured as a generic character. Includes functionality including jumping, running, ducking and movement easing.
  - Controller: This is what possesses and controls a Pawn or Character. The two main varieties are PlayerController and AIController.

CHARACTER OVERVIEW

- In Unreal, a Character component is a fully formed movement component that includes movement, jumping and many other common movement properties.
  - Pros:
    - Easy to get started with
    - Includes lots of functionality (jumping, ducking, integration with pathfinding)
  - Cons:
    - Harder to customize later
    - Much more complex
    - May include unneeded functionality

PAWN OVERVIEW

- In Unreal, a Pawn component is a very simple component used to apply movement or create custom characters. While it does have some limited functionality, it requires the user to essentially build it from the ground up.
  - Pros:
    - Highly customizable.
    - Can be used for many different types of character movement.
  - Cons:
    - You have to build it yourself.

ACTOR OVERVIEW

- If you want to create an object that is not player controlled (a door, a tree, etc.) you can use an Actor. An Actor class is an object class that can be added to the world, has a position, rotation and scale, and can be manipulated with most of the same tools we will be using, however it is not controlled by a player.
- We will look at the relationship between Actors and Pawns more in later lectures.
- For now, think of Pawn as an Actor with added functionality to allow the Player to control it.
  - Actor = Non-Player/AI Controlled Game Objects
  - Pawn = Player/AI Controller Game Objects

CONTROLLER CLASS TYPE

- A controller is essentially the “brain” of a character. It is a non-physical actor that can “possess” a Pawn (or subclass of pawn, such as a Character) and is responsible for controlling its actions.

- See the documentation for full details:
  - https://docs.unrealengine.com/4.27/en-US/InteractiveExperiences/Framework/Controller/

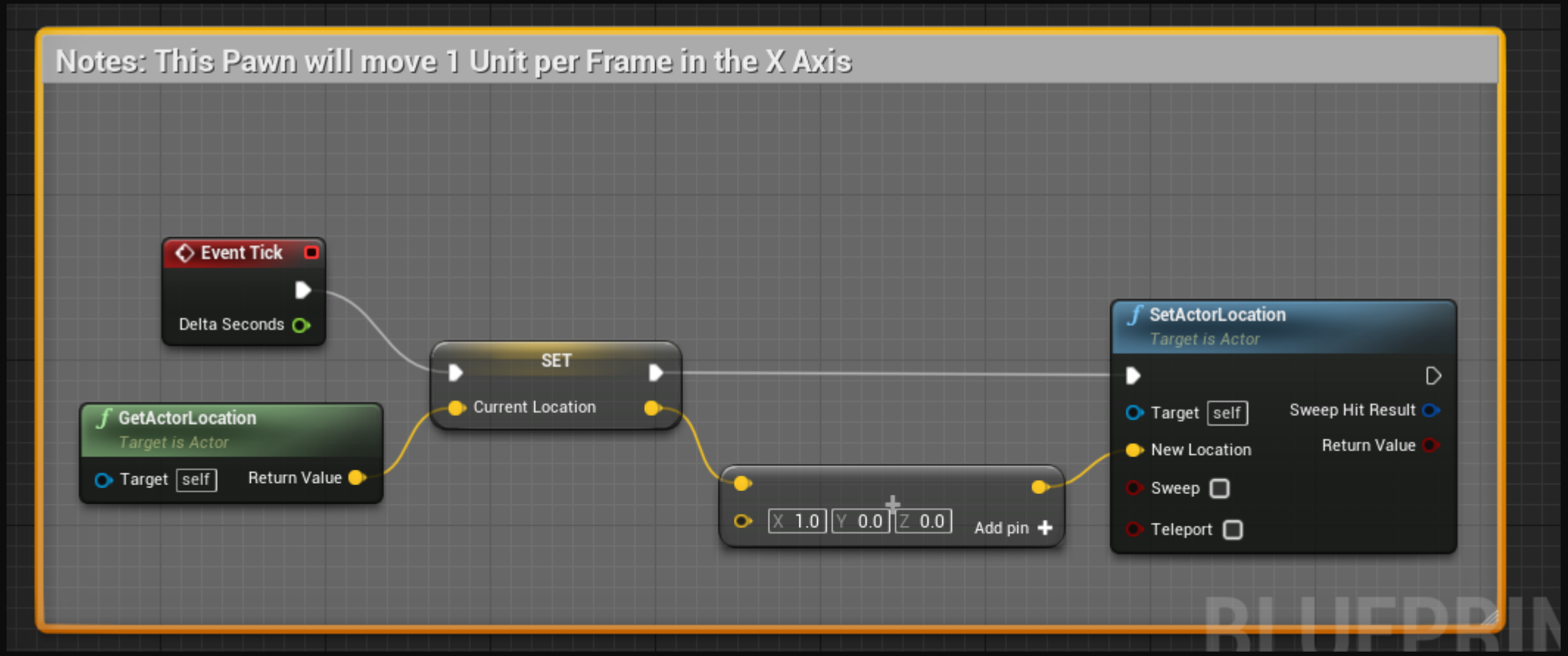
MANUAL MOVEMENT OF A PAWN

- In this course, we will be exploring how to create a custom Pawn component that can take our input (or the input of an AI Controller) and navigate the game world.
- In order to do this, we need to understand how movement works in a modern game engine.
- We will look at Vectors, Rotations and different ways to adjust both of these.

MANUAL MOVEMENT OF A PAWN

- A Position is stored as a 3D Vector that represents the current position of the Pawn in the world. For example, a world position of (0,0,0) would mean it is at the origin of the world. This represents the (X, Y, Z) coordinates of our Pawn.
- This is also known as the Cartesian Coordinate System.

MATH PREREQUISITES – WHERE AM I?



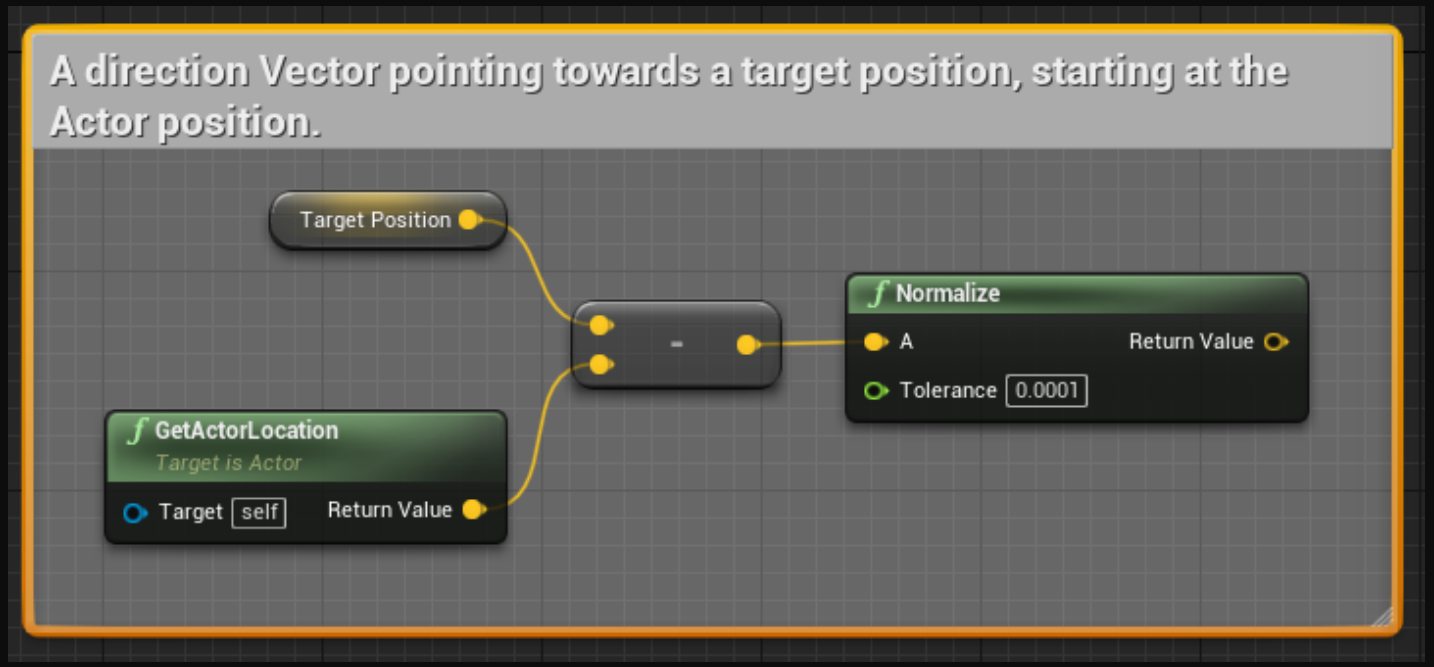
A VECTOR AS A 3D POSITION

- Traditionally in physics, a vector is a direction with a magnitude. In computer science and math, vectors get used in a more general purpose way.
  - For example, Unreal stores the coordinates of a given game object in a Vector.

MOVEMENT IN GAMES

- By incrementally changing the value of a position vector over a sequence of frames, we will give the “Illusion” of smooth movement, just like a movie is a sequence of photographs that give the illusion of movement.
- With this simple concept in mind, we are ready to start creating our own character avatars using nothing but math.

DIRECTION VECTOR TO TARGET



MANUAL MOVEMENT OF A PAWN

- For example, we can add a vector of (1,0,0) to our current position of (0,0,0) each frame to move our avatar.
- We would say our avatar is moving 1 unit per frame in the X direction.

QUICK REVIEW OF VECTOR ADDITION

- Quick review of Vector Addition:
  - When adding two vectors, we add (X1,Y1, Z1) to (X2, Y2, Z2) to get (X3, Y3, Z3) by the following method:

(X1 + X2, Y1 + Y2, Z1 + Z2) = (X3, Y3, Z3)

That is to say, we add the X elements together, then the Y element, then the Z, and are left with a vector solution.

e.g. (0,1,2) + (3, 4, 5) = (0+3, 1+4, 2+5) = (3,5,7)

"COMPUTER SCIENCE" VECTORS OVERVIEW

- A Vector in physics is a quantity that has both a magnitude and a direction.
- However, a Vector data type in computer science can be thought of as a variable that holds a set of values, such as (1,2,3)
- The dimension of a vector is the number of values it contains, such as a Vector2 (X, Y) which is a two dimensional vector. A Vector3 (X, Y, Z) has three dimensions. You can have vectors of any number of dimensions, however these are the most common for our purposes.
- We will use vectors both in the physics sense and as a more general way to store data that is related to each other.

DIFFERENT VECTORS OVERVIEW

- For the purposes of this course, we will be dealing with different types of Vectors, so for clarity I will refer to them as follows:
  - Positional Vector (X,Y,Z) – Stores the position of an Object in the game world using Cartesian Coordinates. Units are in Unreal Units.
  - Rotational Vector (X, Y, Z) – Stores 3 rotational axes that define a rotational orientation. Uses Euler Angles.
  - Directional Vector (X, Y, Z) – Stores a direction and magnitude, this is the Physics definition of a Vector.
    - Note, if you are working in 2D in Unreal, you still have 3D Vectors, you will just set some axis to 0 and ignore it.

POSITIONAL VECTOR

- Positional Vector (X,Y,Z) – Stores the position of an Object in the game world using Cartesian Coordinates. Units are in Unreal Units.
- Game objects in the world have values for X, Y and Z that determine where they are in 3D space.

DIRECTION VECTORS

- Directional Vector (X, Y, Z) – Stores a direction and magnitude, this is the Physics definition of a Vector.
  - Note: This is not the same as a Rotation, which stores the angles of each of the 3 axis in degrees (an Euler angle).

DIRECTIONAL VECTORS

- We have already seen the concept of using a vector to store the current position of something. An object at a location of (10, 5, 3) would be 10 units on the X axis, 5 units on the Y axis and 3 units from the world origin in unreal.
- In unreal, the typical axis's are as follows:
  - the X axis is Forward/Backward (Forward being positive, Backwards negative values)
  - The Y Axis is Left/Right (Right being positive, left negative values)
  - The Z Axis is Up/Down (Up being positive, Down being negative)

    - Be careful because most other 3d suites are different both in which direction each represents, and in the relative orientation to each other. This is why importing assets from 3D tools can be so complicated as they often have to have this orientation corrected.
    - https://www.techarthub.com/a-practical-guide-to-unreal-engine-4s-coordinate-system/

DIRECTIONAL VECTORS

- It should be noted that even though a direction vector represents a direction, it only carries information about 2 Axes of a corresponding rotation, so care must be taken when creating Rotations from directional vectors.
  - We will discuss this later, but the axis that rotates around the direction vector itself cannot be inferred from the direction vector only.
  - Think about this for a while, it makes sense once you understand directional vectors and rotations.

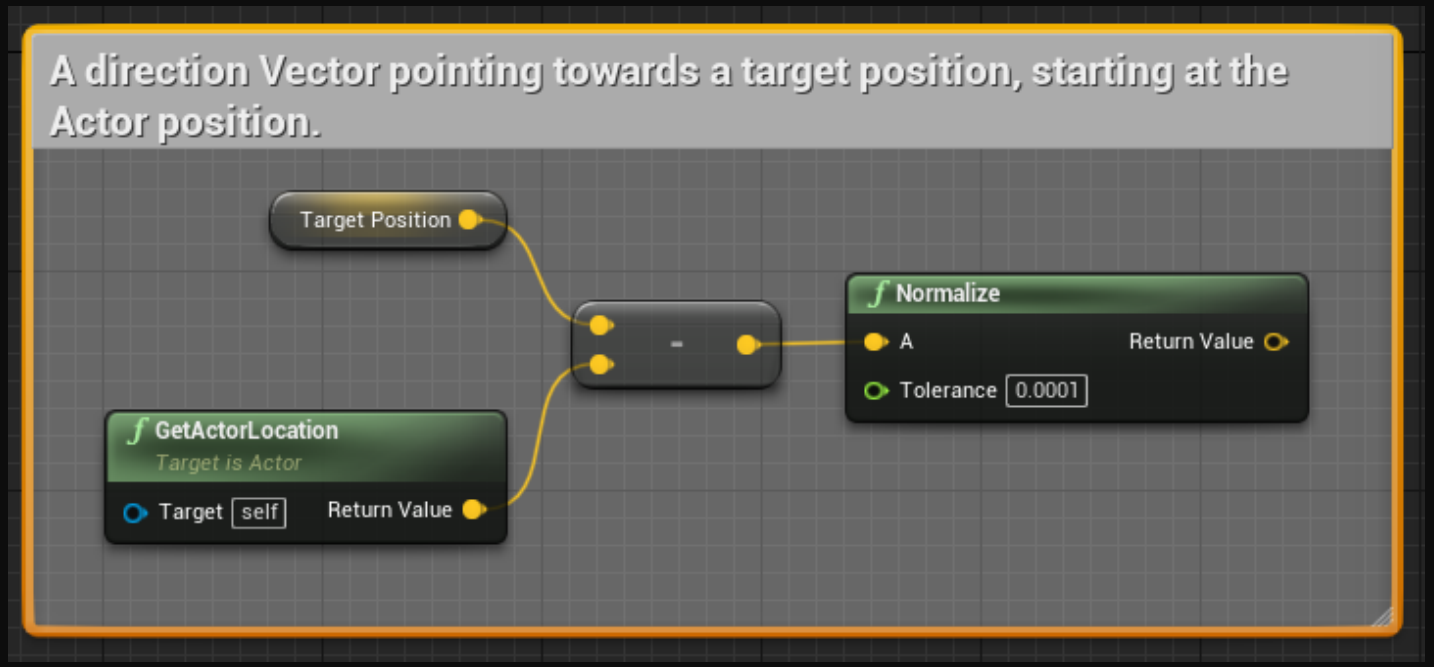
DIRECTIONAL VECTORS

- While we can use vectors for positions, we can also consider a vector as a "direction" itself.
- This is the closest to the idea of a Vector in the field of Physics, where a Vector refers to a direction with a magnitude.
- For example, a vector (1, 0, 0) would be considered a vector with a magnitude of 1, pointing in the +X direction.
- Adding this direction vector to a position vector will give us a new position vectors (our updated position).

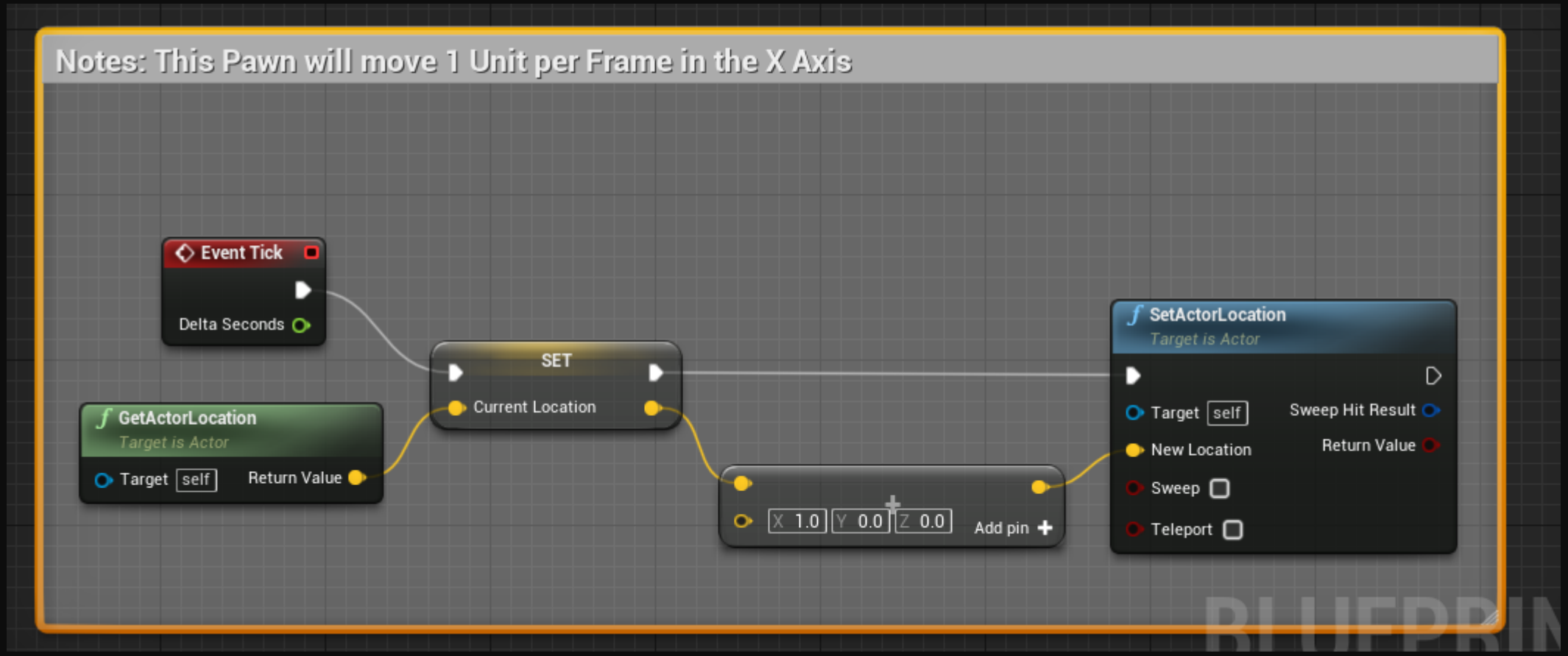
definition:

- Magnitude: Length of the vector as a scalar value (non vector value).

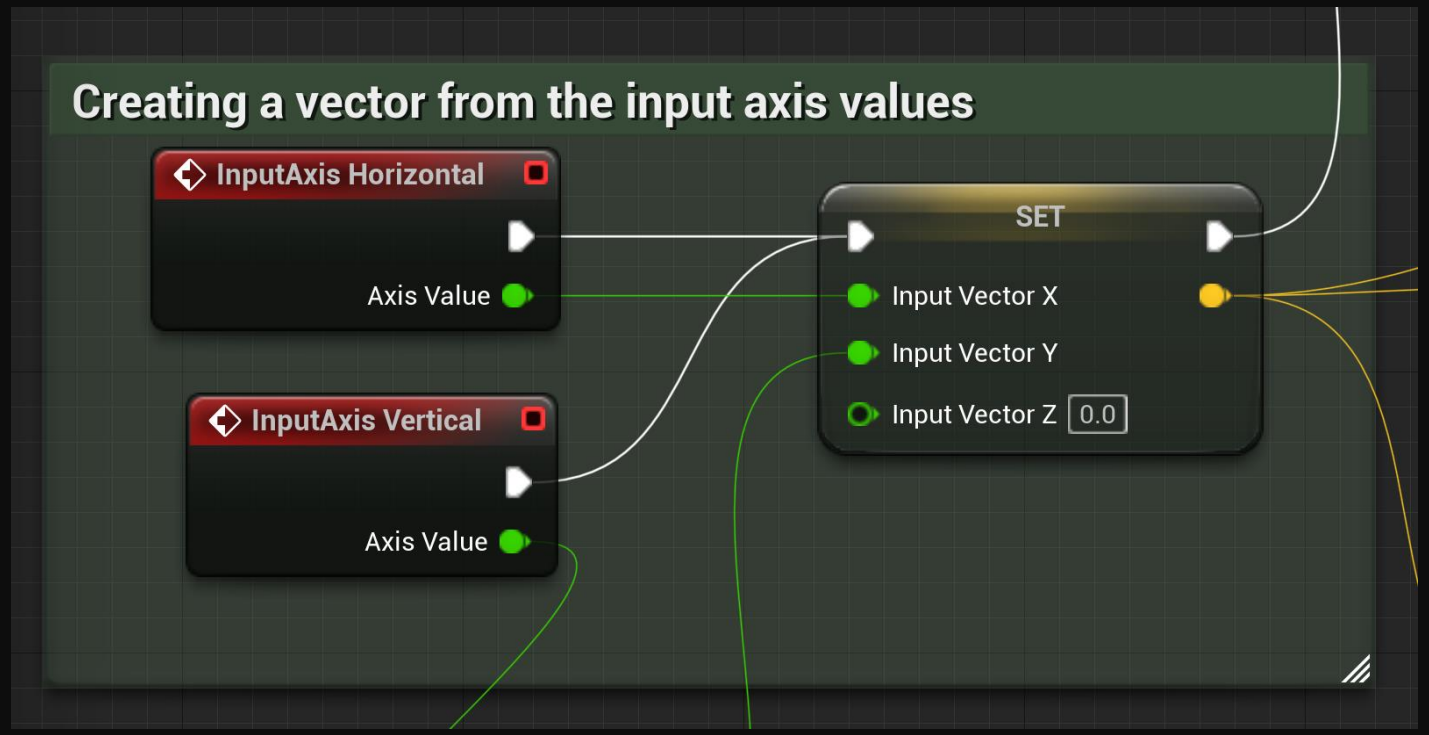
DIRECTION VECTOR TO TARGET



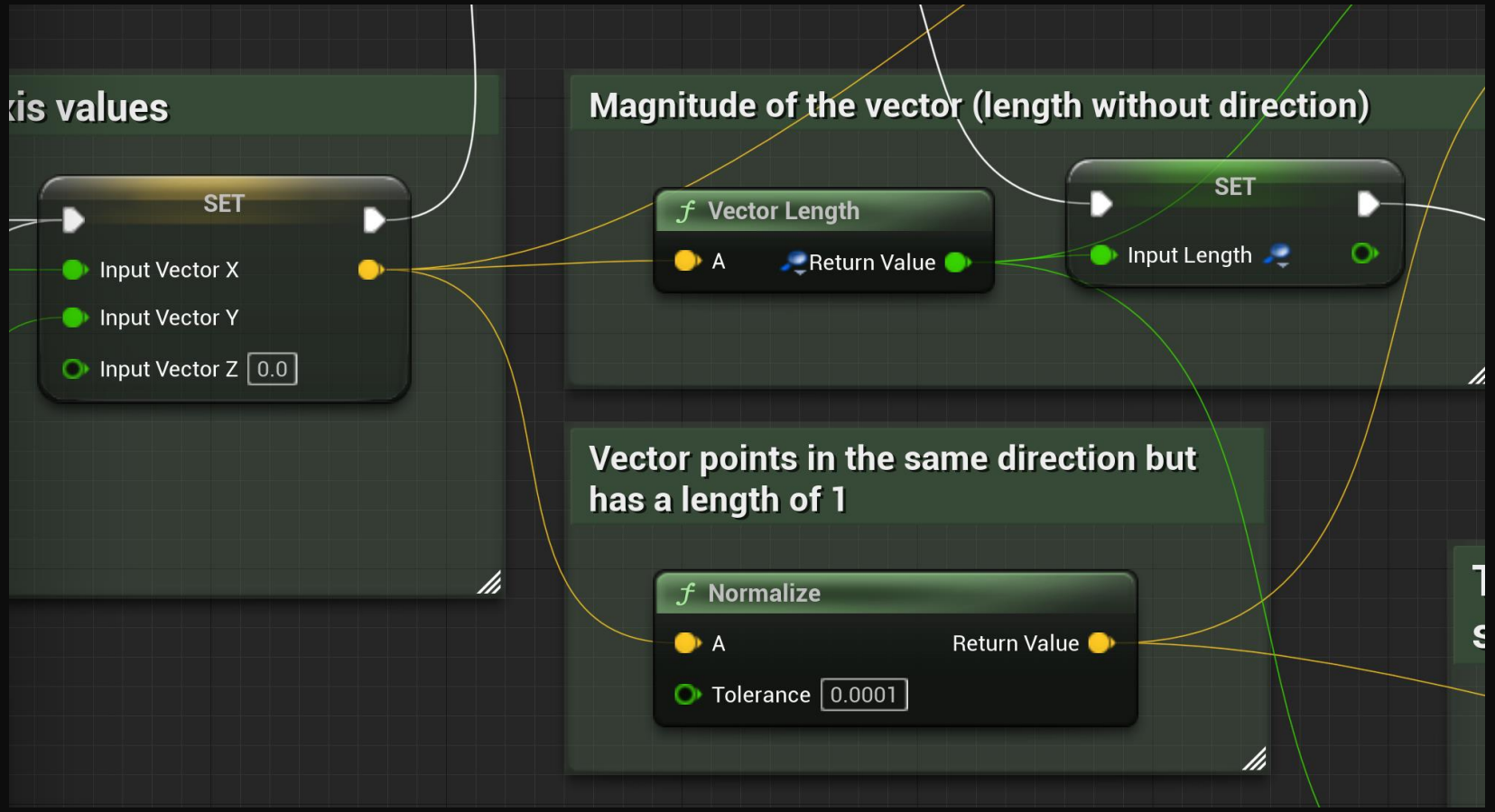
MATH PREREQUISITES – WHERE AM I?



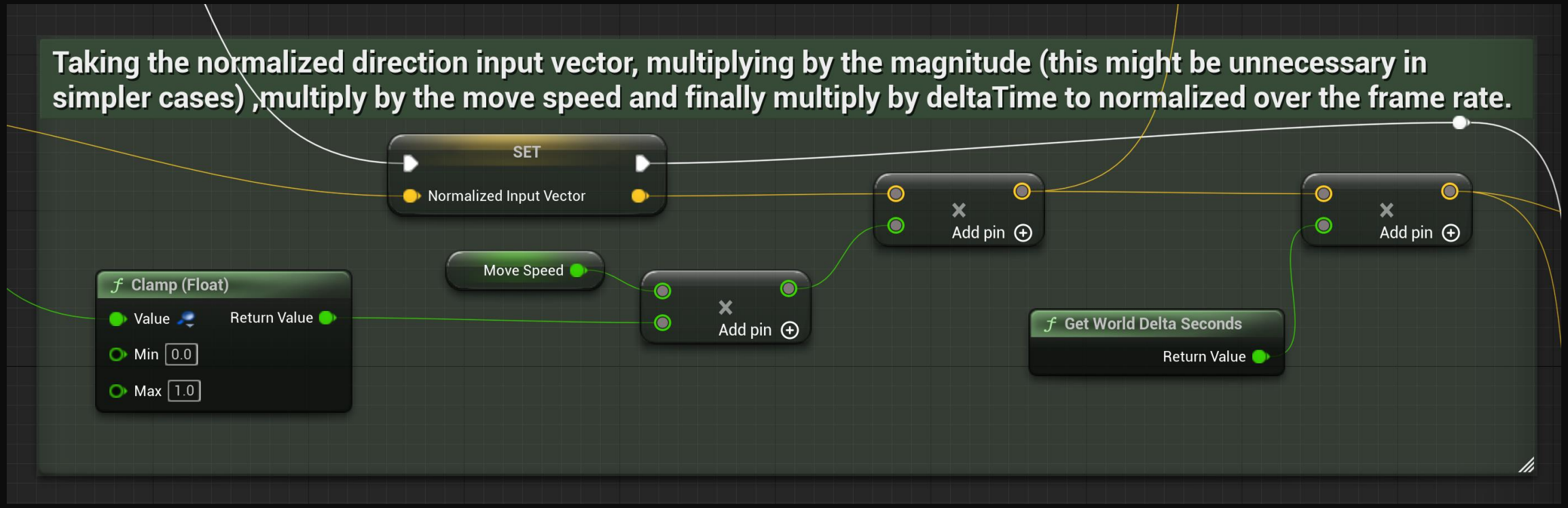
EXAMPLE OF INPUT TO MOVEMENT



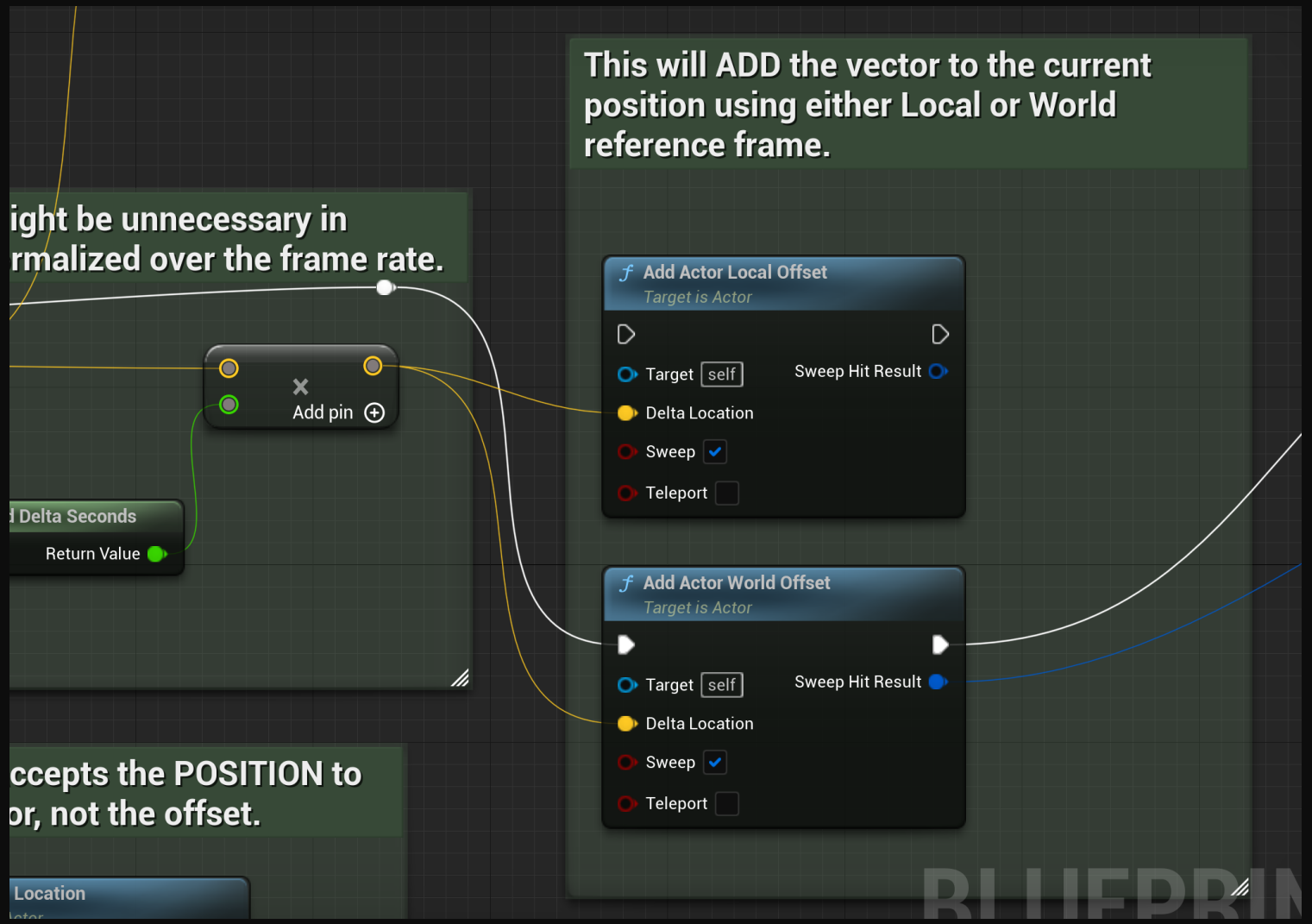
EXAMPLE OF INPUT TO MOVEMENT



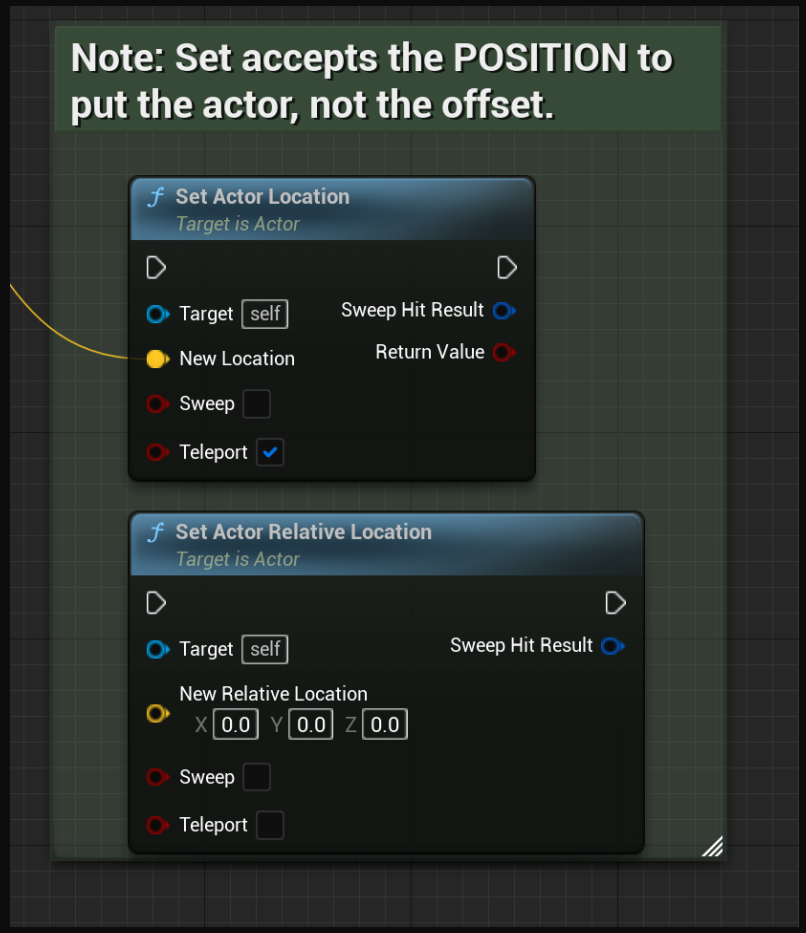
EXAMPLE OF INPUT TO MOVEMENT



EXAMPLE OF INPUT TO MOVEMENT

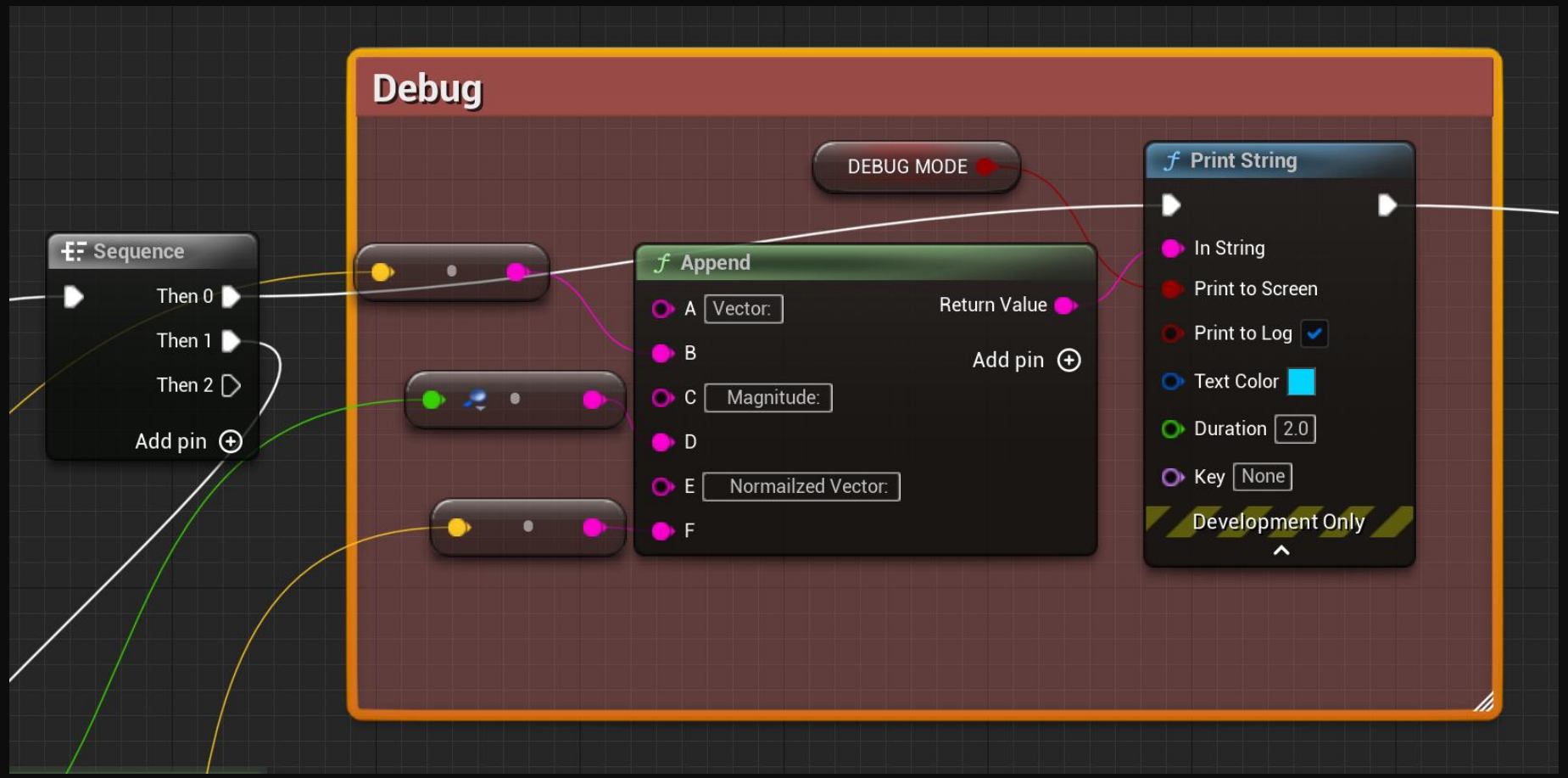


EXAMPLE OF INPUT TO MOVEMENT – EXTRA NODES

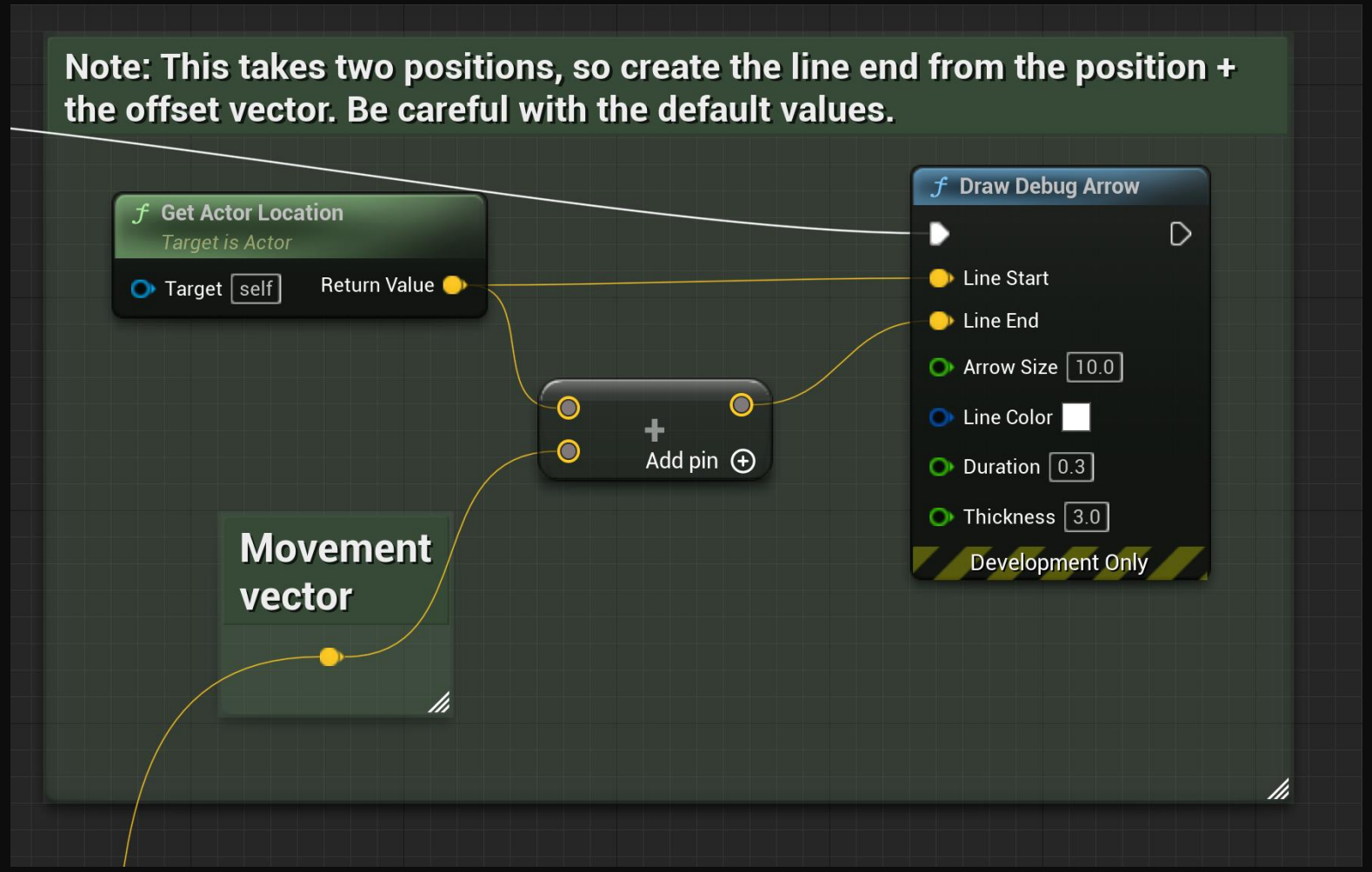


- This is an example of Setting the position directly, rather then offsetting. For this, you would have to calculate the exact position you want the actor first. This is more useful for placing objects directly into the scene. or moving them to fixed locations.

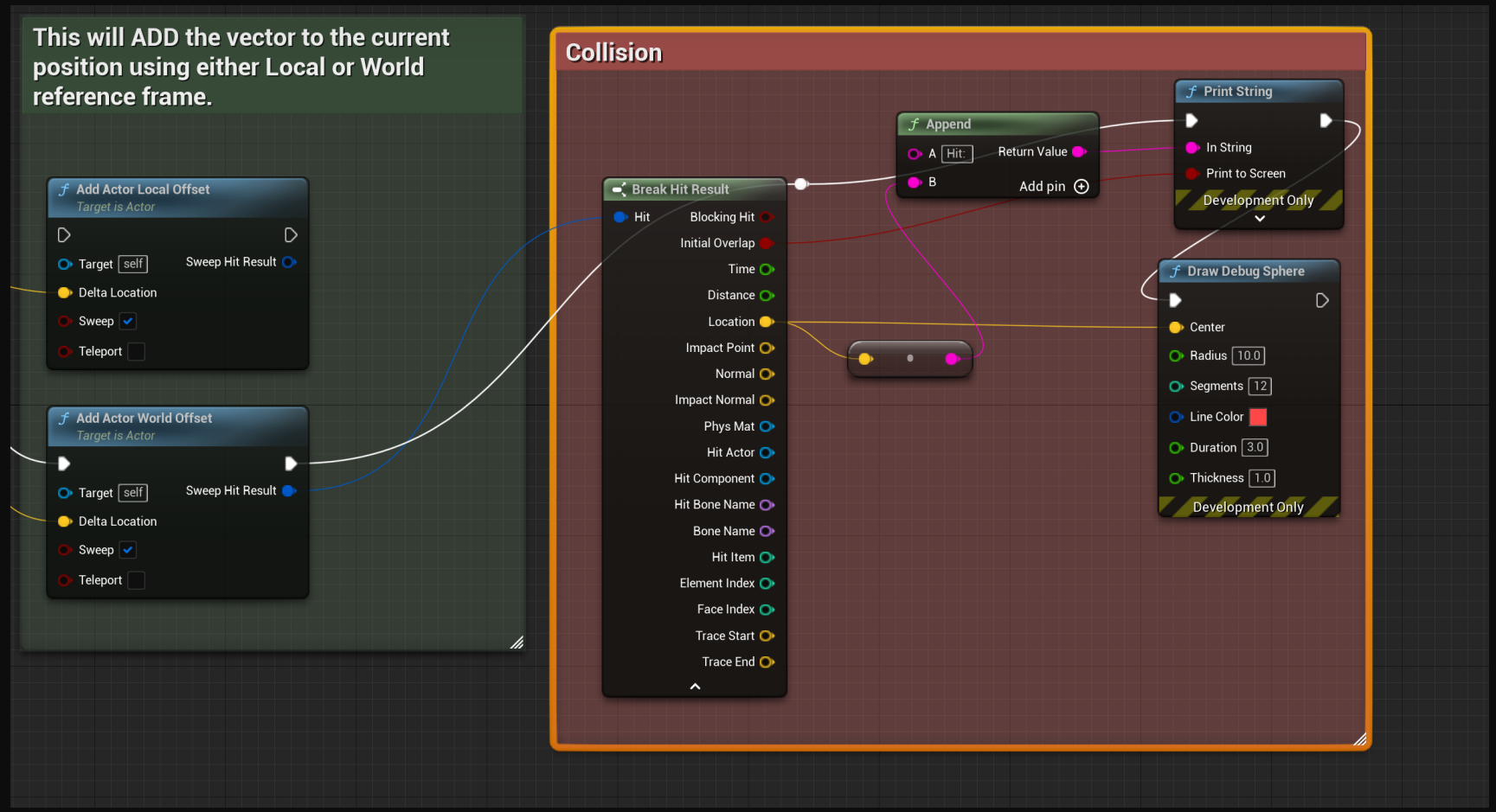
EXAMPLE OF INPUT TO MOVEMENT – DEBUGGING



EXAMPLE OF INPUT TO MOVEMENT – DEBUGGING



EXAMPLE OF INPUT TO MOVEMENT – DEBUGGING



Note : May not be working correctly

DIRECTION VECTORS

- A direction vector can also be manipulated to providing only the direction or the magnitude. Normalization and finding the magnitude are two very useful tools when working in games.
- Normalization: If you set the length of the vector to 1, without changing the direction it points in, you have Normalized the vector. This will return a new vector, pointing in the same direction, with a total length of 1 (combining all 3 axes).
  - Normalization is very useful for navigation and we will see it frequently.
- Magnitude: If you want to find only the length of the vector and discard the directional data, you can find the magnitude of the vector.
  - Note: The Magnitude will be returned as a Scalar value, rather then a Vector.

DIRECTION VECTORS: NORMALIZATION

- Normalization: If you set the length of the vector to 1, without changing the direction it points in, you have Normalized the vector.
- If you take some Vector A, and normalize it to become Vector B, Vector B will be pointing in the same direction, however the sum total length of all three axes will be 1.

DIRECTION VECTORS: MAGNITUDE

- Magnitude: If you want to find only the length of the vector and discard the directional data, you can find the magnitude of the vector.
  - Note: The Magnitude will be returned as a Scalar value, rather then a Vector.
  - If you find the magnitude of a normalized vector, it will always be 1. Why is this?

ROTATIONAL VECTORS

- Rotational Vector (X, Y, Z) – Stores 3 rotational axes that define a rotational orientation. Uses Euler Angles.
  - Note: This is just a generic Vector datatype of floats that happens to hold Euler values. I am just calling it a rotational vector for clarity within the course.
  - Rotational vectors give the angle or orientation of each axes in degrees.
  - Each axis can have a value of [-180, 180] in Unreal.
  - Putting in a value greater then this will simply mod (%) the value by 360.
    - E.g. Enter a value of 900. It will store this value but as soon as you rotate it, it will get cast back into the range (at 180 in this case).

VECTOR VS SCALAR

- A Scalar value is really just a fancy term for a single value (as opposed to a set of values, like a vector).
- In terms of variables, an int, float, Boolean and even a String could be considered Scalar values.
- When referring to scalar multiplication in this course, we will be considering scalar values that have numeric values (int, float) rather then Boolean or String values.

VECTOR VS SCALAR

- A Scalar value multiplied by a Vector will return another Vector.
- A scalar value multiplied by another scalar value will return another scalar value.

UNREAL MOVEMENT CRASH COURSE

QUOTE FROM HTTPS://DOCS.UNREALENGINE.COM/4.27/EN-US/PROGRAMMINGANDSCRIPTING/PROGRAMMINGWITHCPP/UNREALARCHITECTURE/ACTORS/COMPONENTS/

- **Actor Components** (class UActorComponent) are most useful for abstract behaviors such as movement, inventory or attribute management, and other non-physical concepts. Actor Components do not have a transform, meaning they do not have any physical location or rotation in the world.
- **Scene Components** (class USceneComponent, a child of UActorComponent) support location-based behaviors that do not require a geometric representation. This includes spring arms, cameras, physical forces and constraints (but not physical objects), and even audio.
- **Primitive Components** (class UPrimitiveComponent, a child of USceneComponent) are Scene Components with geometric representation, which is generally used to render visual elements or to collide or overlap with physical objects. This includes Static or skeletal meshes, sprites or billboards, and particle systems as well as box, capsule, and sphere collision volumes.

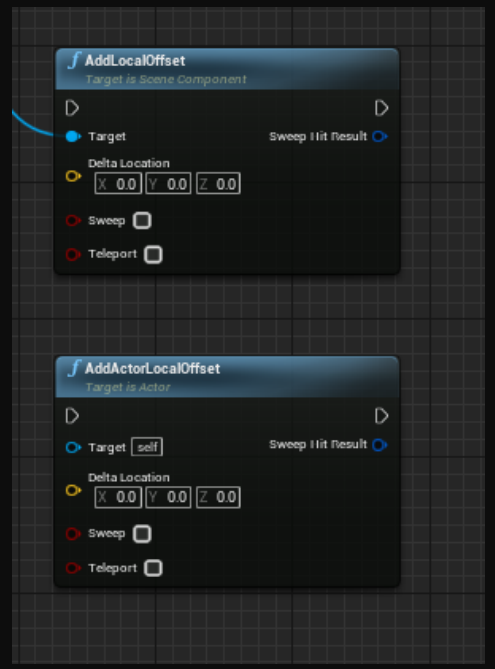
Note that actors do not directly store Transform (Location, Rotation, and Scale) data; the Transform data of the Actor's Root Component, if one exists, is used instead.

https://docs.unrealengine.com/4.27/en-US/ProgrammingAndScripting/ProgrammingWithCPP/UnrealArchitecture/Actors/

WHAT IS DIFFERENT BETWEEN THESE NODES? >??

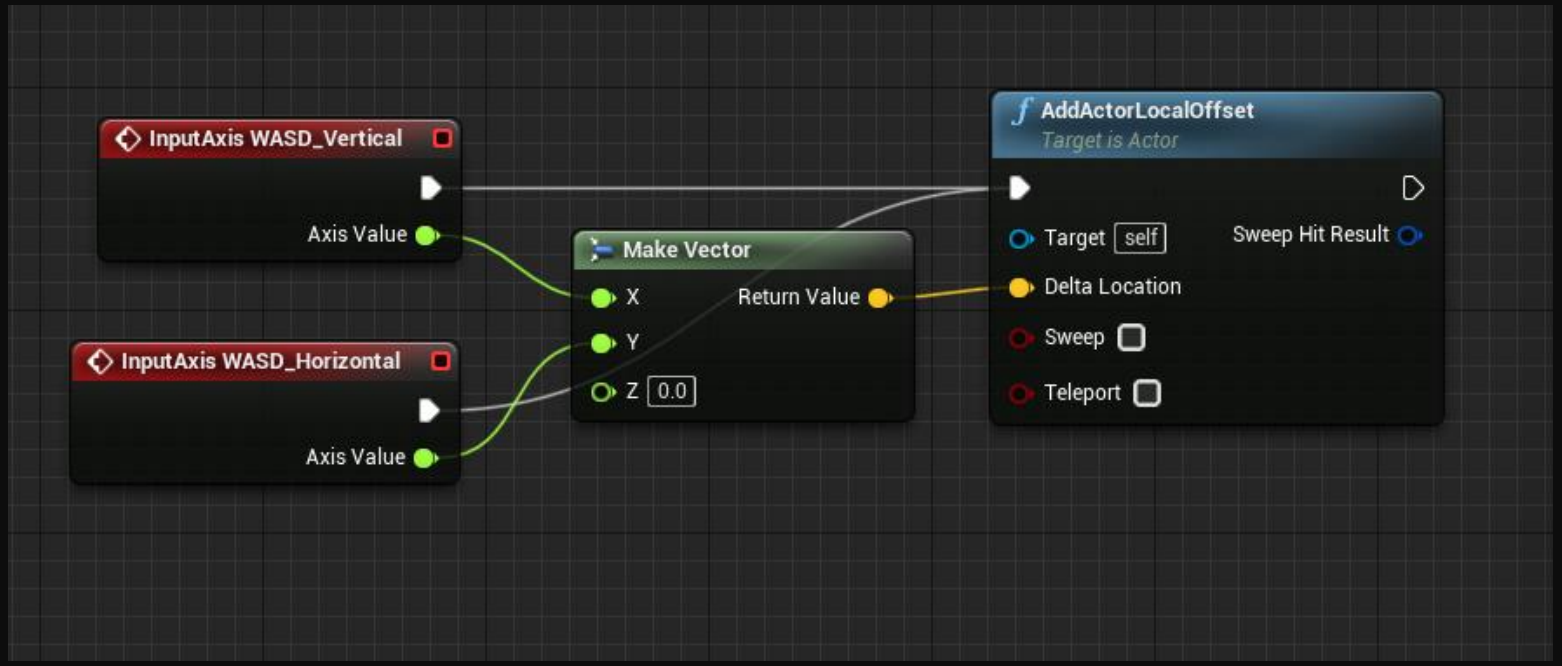
ACTOR TARGETS ACTOR (CAN BE SELF)

ADDLOCALOFFSET NEEDS A SCENECOMPONENT REFERENCE AND MOVES THAT, OTHERWISE THEY ARE THE SAME.



AddActorLocalOffset

This is a simple way to move your Actor around



AddActorLocalOffset

- Breaking down the Node name:
- Add – Usually you will use either Add or Set
  - Add increases the current value by the delta.
  - Set means set the value to the delta provided. It will snap directly that that value.

AddActorLocalOffset

- Breaking down the Node name:
- Actor: Is specifying that this node targets an Actor and will require an actor reference to function.
  - AddLocalOffset is similar but used to move a SceneComponent (such as a Mesh).

AddActorLocalOffset

- Breaking down the Node name:
- Local: Means we are moving in a local coordinate space.
  - Local: For example, adding a vector (1,0,0) to a Pawn in local space will place the Pawn 1 unit in the X direction relative to its own local orientation. That is to say, it will move it forward according to its own forward direction.
  - Adding the same vector in World space would use the Map coordinate system, aka it world move the object in the world X direction, ignoring the Actors
  - Relative: You may also encounter the Relative keyword, which refers to moving/rotating an object Relative to that objects parent transform.
  - It can be hard to see the difference between these nodes until you start manipulating the orientation of them. If they are all behaving the same way, they probably all have the same orientation (that is, there is no rotation applied to the Actor, the parent component or the Local component, so they all act the same).

AddActorLocalOffset

- Breaking down the Node name:
- Offset: Means we are moving the object.
- Alternatives include:
  - Offset: Move the object
  - Rotation: Rotate instead of moving the object
  - Transform: Accept a transform, which includes a Location, a Rotation, and a scale.

AN INCOMPLETE LIST OF NODES FOR REFERENCE

- AddLocalOffset – increments the position of a Scene component in local space
- AddActorLocalOffset – Increments the position of an Actor in Local space
- AddWorldOffset – Increments the positions of a Scene Component in World space
- SetLocalOffset – Sets the local position directly, relative to its parent
- AddActorWorldOffset – Increments the position of an Actor in world space
- SetActorWorldRotation – Sets the rotation of an Actor directly in world space.
-

WHICH DO WE USE?

- The answer is that it depends.
  - Does your movement direction rely on the rotation? Probably local space then.
  - Does your character ignore its rotation and move in fixed directions in the world?
    - Probably world space.

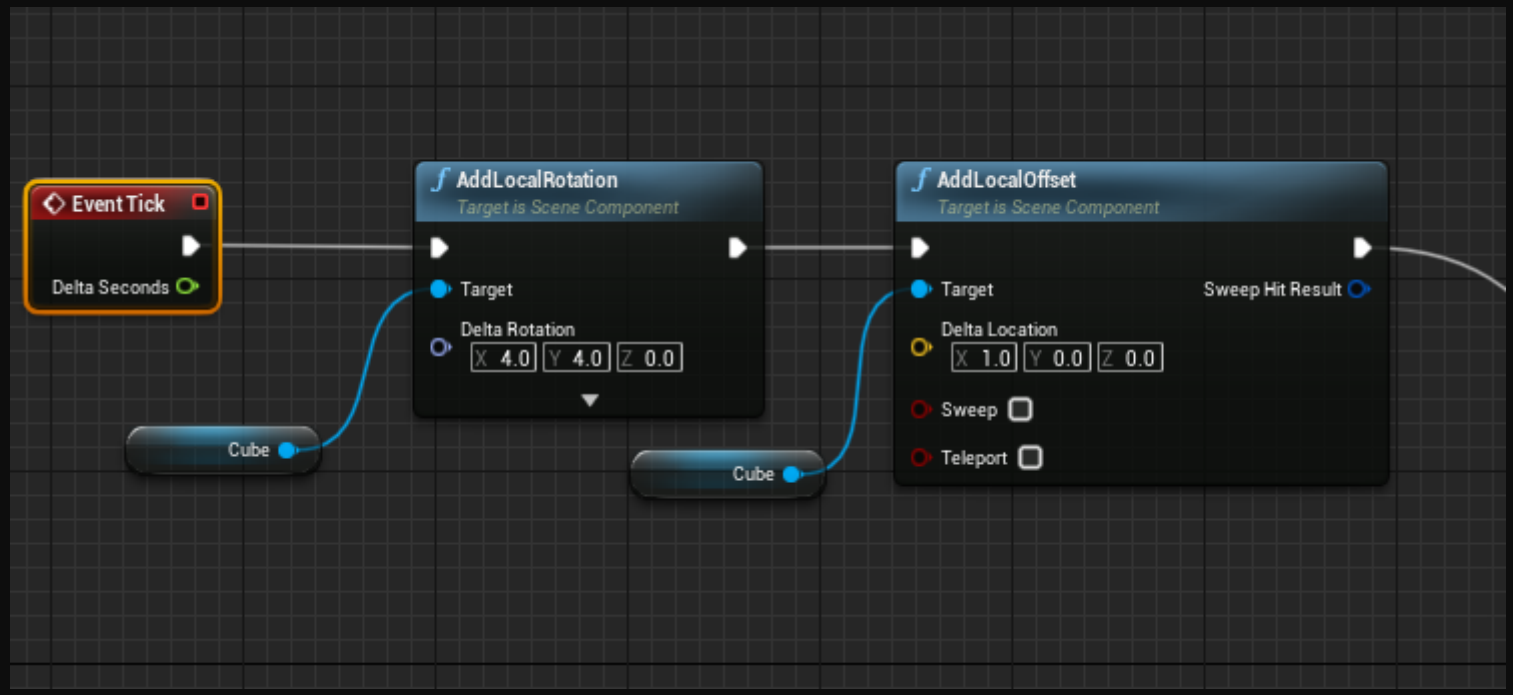
EXERCISE: MOVEMENT AND ROTATION NODES

- Create a pawn and experiment with different combinations of these nodes to compare the effects.

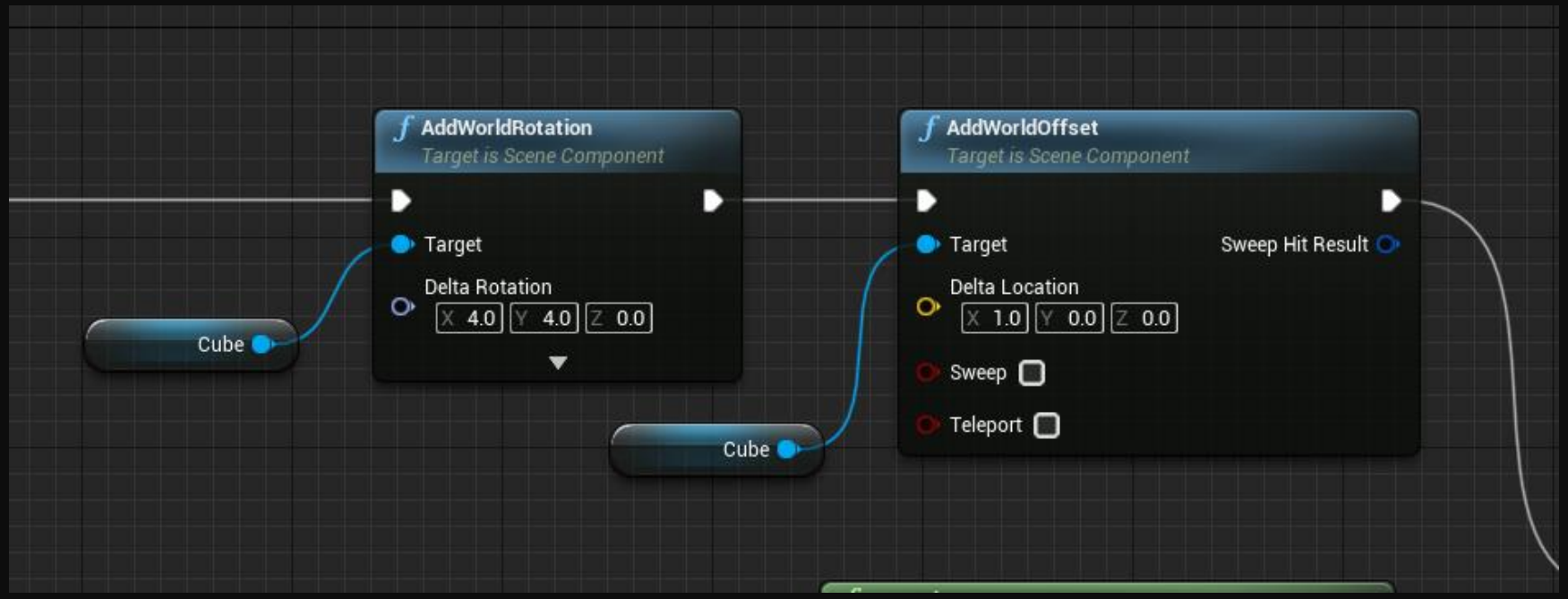
CONCEPT REVIEW – WORLD VS RELATIVE

- Another concept that is important to understand is the difference between World space and Relative Space (Sometime called Local in unreal nodes).
- This is referring to the coordinate system to which the transformation is being applied.
- For example, if I move an Actor using AddWorldOffset, with a value of (1,0,0), it will move in the X direction relative to the coordinate system of the world, ignoring any rotation in the actor itself.
- If I move an Actor using AddLocalOffset, with the same value (1,0,0), it will move in the X direction according to the rotation of the Actor itself. That is, it will move in the X direction of the Actor. If that actor becomes rotated, the direction will change.

CONCEPT REVIEW – WORLD VS RELATIVE

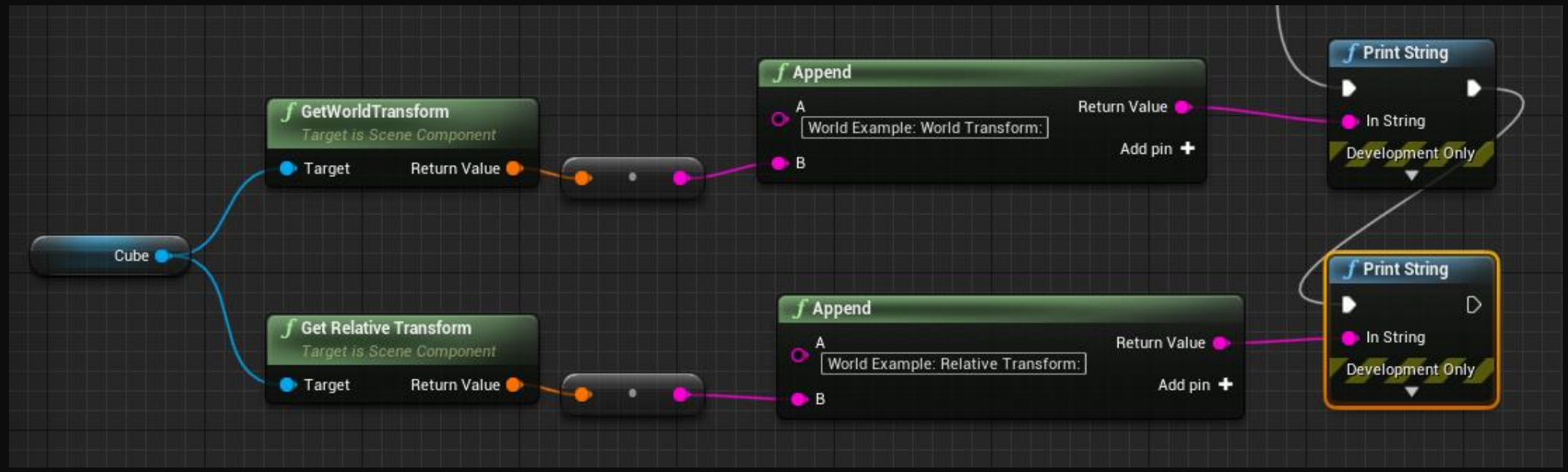


CONCEPT REVIEW – WORLD VS RELATIVE



CONCEPT REVIEW – WORLD VS RELATIVE

- IN THIS EXAMPLE, THE TRANSFORM IS BEING EXTRACTED IN BOTH WORLD SPACE AND LOCAL SPACE. WHAT DIFFERENCES DO YOU NOTICE ABOUT THE OUTPUT? WHAT IS THE SAME?
- IF BOTH OUTPUTS ARE THE SAME, IT PROBABLY MEANS THE ACTOR IS AT THE ORIGIN OF THE WORLD. WHAT HAPPENS IF YOU ROTATE AND/OR MOVE THE ACTOR AROUND?



DYNAMIC VS KINEMATIC MOVEMENT

- Kinematics: Study of motion without forces
- Dynamics: Study of motion with forces
- In game development, kinematic motion is movement without using the physics system to calculate the position of the game object in the world.
  - This is usually directly controlled by the
- Dynamic motion is movement using forces being applied to the physics system.

DYNAMIC VS KINEMATIC MOVEMENT

- Typically in game development, movement is *usually* controlled in a kinematic manner as developers like to retain full control over the position of the controller in the world
- “True” physics is rare even in games that “appear” to have physics.
- Developers will often simulate dynamic motion by including acceleration calculations when determining the position of the game object, but they are usually setting the position (or velocity) directly, rather then trying to control it using forces.

DYNAMIC VS KINEMATIC MOVEMENT

- Neither is right or wrong and sometimes a mix is used
  - For example, many games will use kinematic movement to navigate, then switch to dynamics for ragdoll simulations.
- Unreal provides an implementation of PhysX Physics Engine to drive its 3D physics interactions, and is in the process of adding Chaos Physics, with a focus on destructible and fracturing assets.
- There are other physics systems that can be integrated, such as Havok.
- Usually dynamic physics is used in games that are using ragdoll physics, real-time destruction or use it in a curated way (Totally Accurate Battle Simulator is a good example of this).

DYNAMIC VS KINEMATIC MOVEMENT

- For our purposes in this course (focusing on movement)
- Kinematic movement will mean we are setting the position directly and will not include accurate acceleration and velocity calculations.
- Dynamic movement will mean we are moving by adding forces (and stopping by adding drag) using the physics engine.
- I might also talk about "Kinematic Physics" or "Kinematic Dynamics", by which I mean a system where we are *loosely* simulating physics movement by calculating the velocity from an acceleration value ourselves, however we are setting the position of the object directly, rather then applying forces (using the physics system). This is the most common way of supporting physics in games.

DYNAMIC VS KINEMATIC COLLISION

- This is further confused by collisions, which can happen to either kinematic or dynamic physics systems, depending on the game engine. For the purposes of our discussion, the previous slides are all referring to the movement of the character.
- Collision detection is a separate but related topic that we will not be covering in great detail.